HW2. Computing with CCs

10.10.25

I. CC Arithmetic

Starting with **the three basic logic gates only**, design CCs that compute the following functions. At any stage you may define a new boxed circuit from a previously established circuit or logic gate. Useful circuits to define in this way include **All-1**, **XOR**, **half-adder**, and (if you wish) **full-adder**. You will also want to introduce boxed circuits for specific mathematical functions as the need arises.

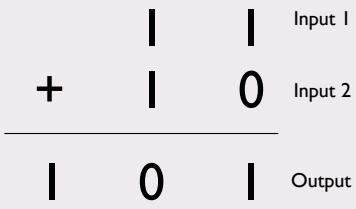
1. +1 [0-15] B
2. +2 [0-15] B
3. +3 [0-15] B
4. $2x$ [0-15] B
5. $2x$+1 [0-15] B
6. $2(x+1)$ [0-15] B (only use *one* sub-part which computes +1)
7. $x+y$ [0-15] B (i.e. addition, bounded to 0-15 for each argument) *
   Reference**:** Petzold (2000) "A binary adding machine."
8. For any integer n, in principle, could you design a CC that computes +1 [0- n] B? Why or why not? Short paragraph.
9. Could you design a CC that computes +1 B (i.e. unbounded)? Why or why not? Short paragraph.

Challenge problem: prove the following: for any numerical function f and any integer n, there is a CC which computes f [0 - n] B.

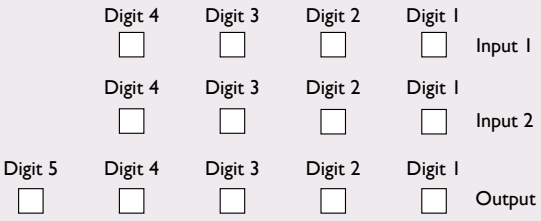
On binary addition

Hint 1: Use the grade school algorithm for long addition.

Start by thinking of the problem in terms of long addition, e.g.:



Then set up your machine in the same format:

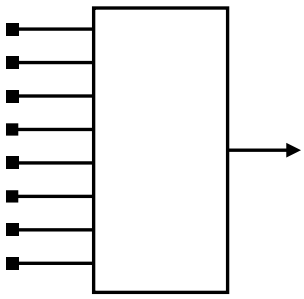


When filling in the circuit, think of your machine first in terms of columns (e.g. how does the Output Digit 1 relate to the Input Digit 1's?). Then think about how the columns relate to each other. A key idea here is the "carry digit"– remember how this works in standard long addition, and see if you can apply the same idea to the construction of your machine.

Hint 2: Add twice. When you are doing long addition, note that you really have to do basic addition *twice* for each column. First you have to add the digit from the top row to the digit from the bottom row, and then you have to add the carry to the result. (You can also do these in the opposite order.) The same thing is true in binary, a fact you should convince yourself of before trying to solve the problem. You'll find this idea has a direct correlate in the circuit which computes binary addition.

II. CC Perception

For each of the following problems assume a CC that has 8 inputs and 1 output, like the schematic circuit below. Think of these as computing **perception**: they take as their input an array of stimulations (like hitting the retina) and they output a symbol which represents a specific shape or category.



10. **Edge detector.** Design a machine that obeys the following rule: output 1 iff there is a string of *at least* three consecutive 1's anywhere in the input.

11. **Object detector.** Design a machine that obeys the following rule: output 1 iff there is a string of *exactly* three consecutive 1's anywhere in the input.

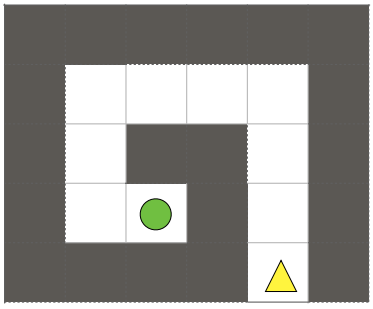
12. **Landmark recognition.** Design a machine that obeys the following rule: output 1 iff the input = 110010111

III. CC Navigation

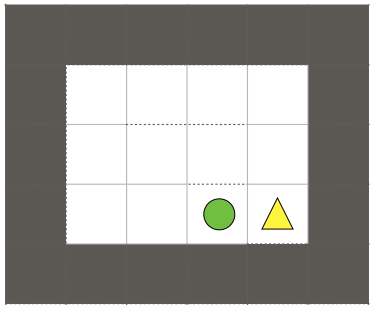
In these navigation tasks, the yellow triangle indicates the turbot's starting position and orientation. Navigation is "completed," and the problem solved, when the turbot *crosses over* the goal. It does not need to stop on the goal. (Think of Pac-Man.)

For each of the following problems, answer the question: can it be completed by a combinatorial circuit embedded in a turbot? If so, give the circuit. If not, explain why in a short paragraph.

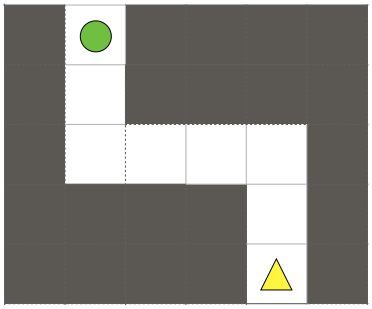
13. **Spiral**



14. **Full circle**



15. **Zig-zag**



IV. Reflection

16. **Representational systems.** Suppose you were designing a calculator for solving a high-stakes problem. (Perhaps "you" are Nature, the "calculator" is the Brain, and the "problem" is Survival.) Would you design it to calculate in binary or in tally? Why? Use specific examples to justify your answer. ~1 paragraph

17. **Embodied computation.** Consider a Turbot like the one in problem 12. What function, if any, does it compute? If it does compute a function, what are the arguments and what is the value of this function? How does this compare to the function computed by the "directions" mode in Google Maps? ~1 paragraph